
3.2.16 RESUMEN DE SECCIÓN

1. Existen dos tipos de bucles en Python: `while` y `for` :

- El bucle `while` ejecuta una sentencia o un conjunto de sentencias siempre que una condición booleana especificada sea verdadera, por ejemplo:

```
1  # Ejemplo 1
2  while True:
3      print("Atrapado en un bucle infinito.")
4
5  # Ejemplo 2
6  counter = 5
7  while counter > 2:
8      print(counter)
9      counter -= 1
10
```

- El bucle `for` ejecuta un conjunto de sentencias muchas veces; se usa para iterar sobre una secuencia (por ejemplo, una lista, un diccionario, una tupla o un conjunto; pronto aprenderás sobre ellos) u otros objetos que son iterables (por ejemplo, cadenas). Puedes usar el bucle `for` para iterar sobre una secuencia de números usando la función incorporada `range` . Mira los ejemplos a continuación:

```
1  # Ejemplo 1
2  word = "Python"
3  for letter in word:
4      print(letter, end="*")
5
6  # Ejemplo 2
7  for i in range(1, 10):
8      if i % 2 == 0:
9          printprint(i)
10
```

2. Puedes usar las sentencias `break` y `continue` para cambiar el flujo de un bucle:

- Utiliza `break` para salir de un bucle, por ejemplo:

```
1 | text = "OpenEDG Python Institute"
2 | for letter in text:
3 |     if letter == "P":
4 |         break
5 |     print(letter, end=" ")
6 |
```

- Utiliza `continue` para omitir la iteración actual, y continuar con la siguiente iteración, por ejemplo:

```
1 | text = "pyxpyxpyx"
2 | for letter in text:
3 |     if letter == "x":
4 |         continue
5 |     print(letter, end=" ")
6 |
```

3. Los bucles `while` y `for` también pueden tener una cláusula `else` en Python. La cláusula `else` se ejecuta después de que el bucle finalice su ejecución siempre y cuando no haya terminado con `break`, por ejemplo:

```
1 | n = 0
2 |
3 | while n != 3:
4 |     print(n)
5 |     n += 1
6 | else:
7 |     print(n, "else")
8 |
9 | print()
10 |
```

```
11 | for i in range(0, 3):
12 |     print(i)
13 | else:
14 |     print(i, "else")
15 |
```

4. La función `range()` genera una secuencia de números. Acepta enteros y devuelve objetos de rango. La sintaxis de `range()` tiene el siguiente aspecto: `range(start, stop, step)` , donde:

- **start** es un parámetro opcional que especifica el número de inicio de la secuencia (0 por defecto)
- **stop** es un parámetro opcional que especifica el final de la secuencia generada (no está incluido).
- y **step** es un parámetro opcional que especifica la diferencia entre los números en la secuencia es (1 por defecto.)

Código de ejemplo:

```
1 | for i in range(3):
2 |     print(i, end=" ") # output: 0 1 2
3 |
4 | for i in range(6, 1, -2):
5 |     print(i, end=" ") # output: 6, 4, 2
6 |
```